

ECE 382N-Sec (FA26):

L6: Transient-Execution Defenses

Neil Zhao

neil.zhao@utexas.edu

Spectre = Misprediction + Microarchitectural Side/Covert Channel

Using Flush+Reload

```
void vulnerable_code(size_t idx) {  
    if (idx < array_size) { // mispredict  
        u8 data = array[idx]; // access  
        u8 junk = glb_array[data*4096]; // transmit  
    }  
}
```

Using SMT port contention¹

```
void vulnerable_code(size_t idx) {  
    if (idx < array_size) { // mispredict  
        u8 data = array[idx]; // access  
        if (data) div(); // transmit via  
        else mul();      // port contention  
    }  
}
```

Spectre gadget/Universal Read Gadget
(Leak arbitrary byte from the same address space)



¹Bhattacharyya et al. "SMoTherSpectre: Exploiting Speculative Execution through Port Contention"

Spectre Software Mitigation: LFENCE



```
void vulnerable_code(size_t idx) {  
    if (idx < array_size) {  
        LFENCE; // block spec. access  
        u8 data = array[idx]; // Access  
        LFENCE; // block transmission  
        u8 junk = glb_array[data * 4096]; // Transmit  
    }  
}
```

Stops all instructions,
high performance overhead

“The LFENCE instruction and other serializing instructions ensure that no later instruction will execute, even speculatively, until all prior instructions have completed locally.” from Intel’s Spectre mitigation guidance¹

¹<https://www.intel.com/content/www/us/en/developer/articles/technical/software-security-guidance/advisory-guidance/bounds-check-bypass.html>

Spectre Software Mitigation: Array Index Nospec



```
void vulnerable_code(size_t idx) {  
    if (idx < array_size) {  
        _idx = array_index_nospec(idx, array_size);  
        u8 data = array[idx]; // Access  
  
        u8 junk = glb_array[data * 4096]; // Transmit  
    }  
}
```

A branchless helper MACRO that clamps the index within the range of `[0, array_size)`

Manual effort!

False positives?

Spectre Software Mitigation: Software Load Hardening (SLH)

```
if (idx < array_size) {  
    u8 data = array[idx]; // Access  
    u8 junk = glb_array[data * 4096]; // Transmit  
}
```



Load address hardening

```
1 uintptr_t all_ones_mask = (uintptr_t)0xffffffffffffffff;  
2 uintptr_t mask = 0;  
3 bool pred = idx < array_size;  
4 if (pred) {  
5     mask = CMOV(!pred, all_ones_mask);  
6     u8 *ptr = &array[idx];  
7     ptr |= mask; // mask is all ones if mis-speculates  
8     u8 data = *ptr; // Access; Invalid address 0xffff.. if mispredicts  
9     u8 junk = glb_array[data * 4096]; // Transmit  
}
```

Spectre Software Mitigation: Software Load Hardening (SLH)

```
if (idx < array_size) {  
    u8 data = array[idx]; // Access  
    data += 10;  
    u8 junk = glb_array[data * 4096]; // Transmit  
}
```



Load value hardening
(allows speculative accesses, blocks transmission)

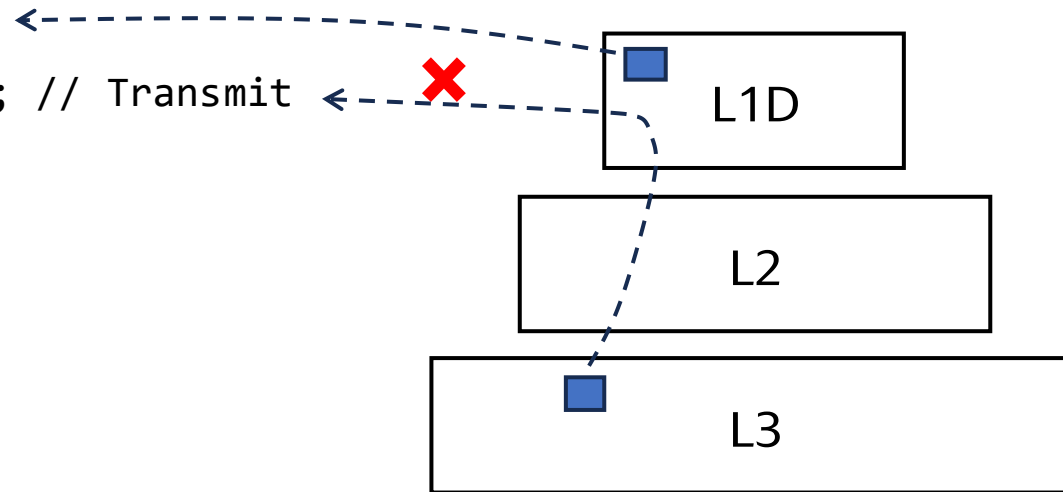
Can speculatively execute
Lower overhead {

```
1 uintptr_t all_ones_mask = (uintptr_t)0xffffffffffffffff;  
2 uintptr_t mask = 0;  
3 bool pred = idx < array_size;  
4 if (pred) {  
5     mask = CMOV(!pred, all_ones_mask);  
6     u8 data = array[idx]; // speculative access  
7     data += 10; // addition is data-oblivious and not a transmitter  
8     data |= mask; // data won't contain any secret if mispredicts  
9     u8 junk = glb_array[data * 4096]; // Transmit  
}
```

Academic Hardware Mitigation: Delay-on-Miss*

- **Basic idea:** Hide the cache side effects during mis-speculation (invisible speculation)
- **Design point:** Speculative loads can access the L1 cache
 - L1 hit $\Rightarrow$ Fetch the data without updating replacement policy states; only update those states when the load becomes non-speculative
 - L1 miss $\Rightarrow$ Stall until the load becomes non-speculative (or predicts its value)

```
if (idx < array_size) {  
    u8 data = array[idx]; // Access  
    u8 junk = glb_array[data * 4096]; // Transmit  
}
```



* Sakalis et al., "Efficient Invisible Speculative Execution through Selective Delay and Value Prediction" (ISCA '19)

Breaking DoM

Use a different covert channel

```
void vulnerable_code(size_t idx) {  
    if (idx < array_size) { // mispredict  
        u8 data = array[idx]; // access  
        if (data) div(); // transmit via  
        else mul();      // port contention  
    }  
}
```

Can we break DOM **under their threat model**
(i.e., only consider leakage due to data loads)

Breaking DoM with Speculative Interference Attacks*

```
1 // Each array entry is 64B (the cacheline size)
2 load &array[0]; // caching line 0
3 flush &array[1]; // flushing line 1
4
5 if ( /* false */ ) {
6     sec = load attacker_ptr; // access; L1 hit; M1
7     data = load &array[sec & 1]; // transmit; M2
8     // whether M2 hits in the L1D leaks the secret
9 }
```

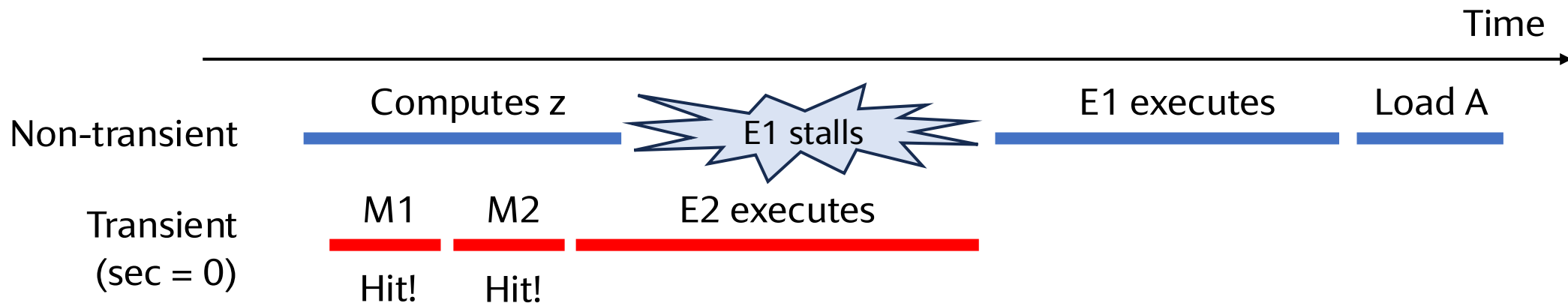
How to detect whether M2 hits in the L1D?
Timing!

* Behnia et al., “Speculative Interference Attacks: Breaking Invisible Speculation Schemes” (ASPLOS ’21)

Breaking DoM with Speculative Interference Attacks

Detecting whether M2 hits in the L1D

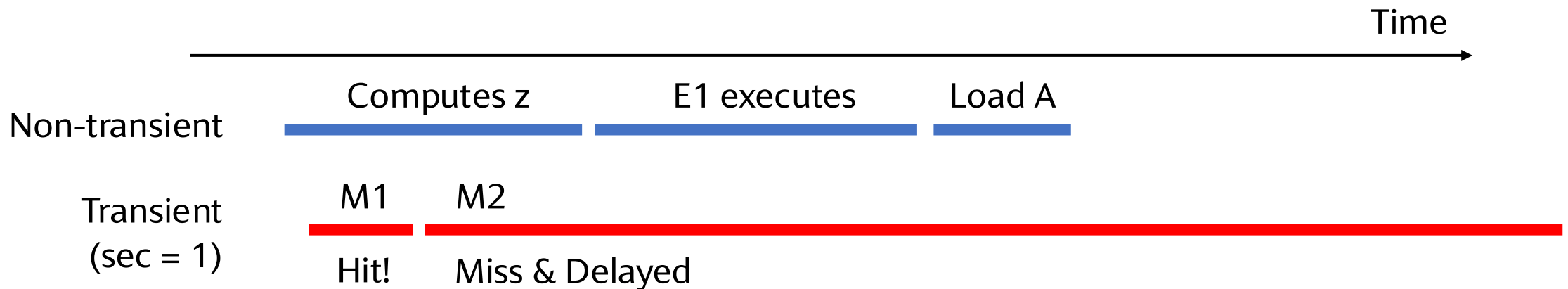
```
1 z = ... // takes Z cycles
2 A = non_pipeline_EU(z); // E1
3 load A; // A non-speculative load that is visible to the attacker
4
5 if ( /* false */ ) {
6     sec = load attacker_ptr; // access; L1 hit; M1
7     data = load &array[sec & 1]; // transmit; M2
8     non_pipeline_EU(data); // E2
9 }
```



Breaking DoM with Speculative Interference Attacks

Detecting whether M2 hits in the L1D

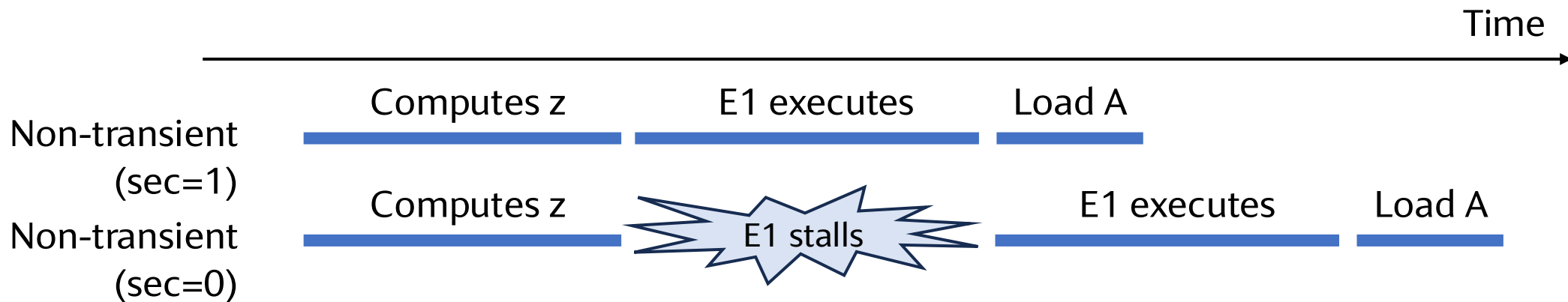
```
1 z = ... // takes Z cycles
2 A = non_pipeline_EU(z); // E1
3 load A; // A non-speculative load that is visible to the attacker
4
5 if ( /* false */ ) {
6     sec = load attacker_ptr; // access; L1 hit; M1
7     data = load &array[sec & 1]; // transmit; M2
8     non_pipeline_EU(data); // E2
9 }
```



Breaking DoM with Speculative Interference Attacks

Detecting whether M2 hits in the L1D

```
1 z = ... // takes Z cycles
2 A = non_pipeline_EU(z); // E1
3 load A; // A non-speculative load that is visible to the attacker
4
5 if ( /* false */ ) {
6     sec = load attacker_ptr; // access; L1 hit; M1
7     data = load &array[sec & 1]; // transmit; M2
8     non_pipeline_EU(data); // E2
9 }
```

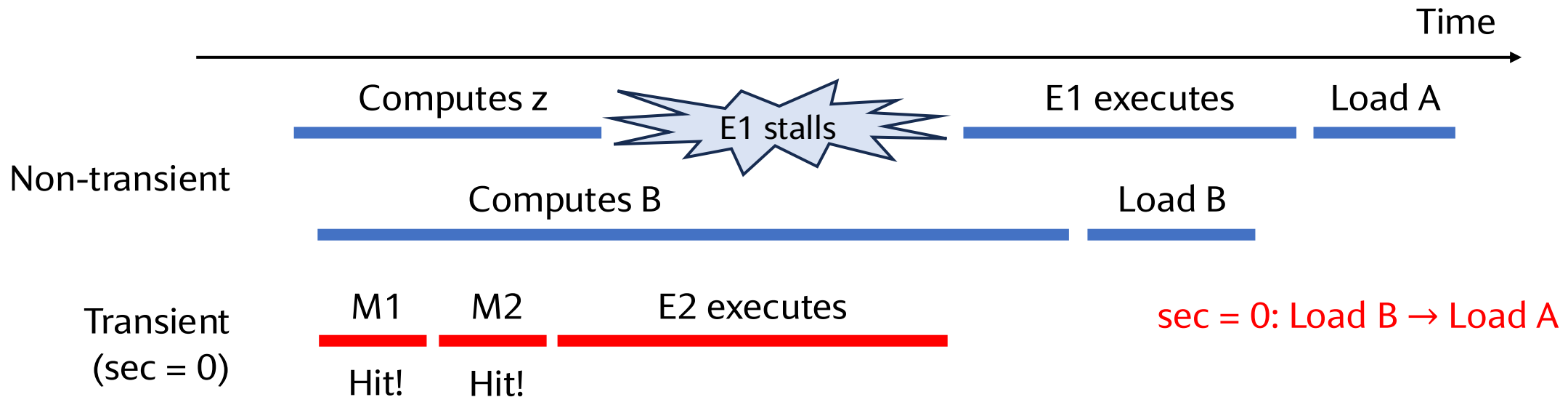


Breaking DoM with Speculative Interference Attacks

Detect the timing of load A

```
1 z = ... // takes Z cycles
2 A = non_pipeline_EU(z); // E1
3 load A;
4 B = ... // takes B cycles
5 load B;
```

```
6 if ( /* false */ ) {
7     sec = load attacker_ptr; // access; M1
8     data = load &array[sec & 1]; // M2
9     non_pipeline_EU(data); // E2
10 }
```

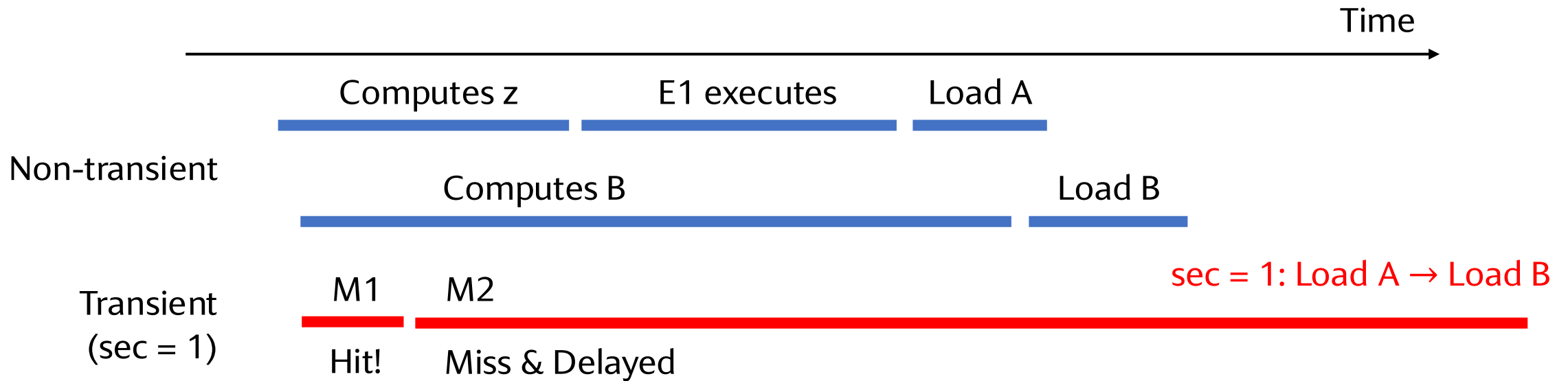


Breaking DoM with Speculative Interference Attacks

Detect the timing of load A

```
1 z = ... // takes Z cycles
2 A = non_pipeline_EU(z); // E1
3 load A;
4 B = ... // takes B cycles
5 load B;

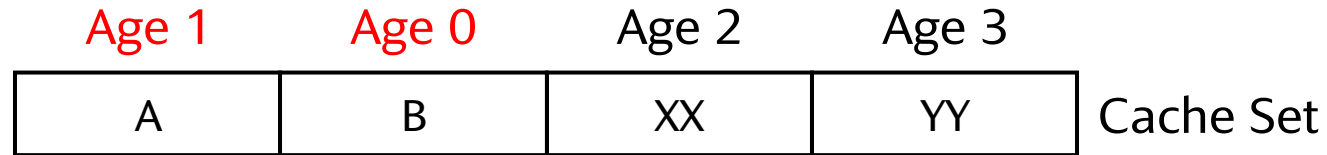
6 if ( /* false */ ) {
7     sec = load attacker_ptr; // access; M1
8     data = load &array[sec & 1]; // M2
9     non_pipeline_EU(data); // E2
10 }
```



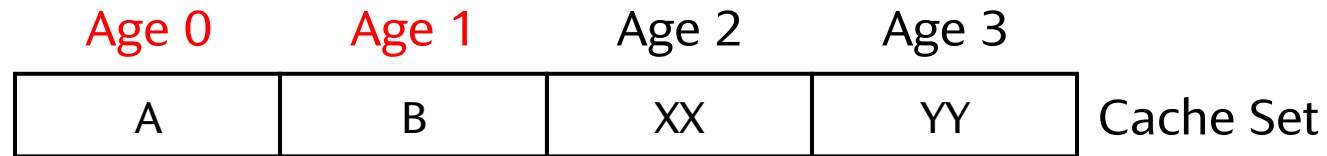
Detecting Load Timing via Replacement Policy State

Detect the order of loads A and B

Load A → Load B



Load B → Load A



Accessing three lines E, F, G that are mapped to the same set

Load A → Load B ⇒ B remains cached ⇒ sec = 1

Load B → Load A ⇒ A remains cached ⇒ sec = 0

Speculative Taint Tracking (STT)*

- **Basic idea:**
 - **Heuristic:** Speculatively accessed data are secrets (that we want to protect)
 - **Guarantee:** Secrets cannot be passed to a speculative **transmitter**
 - Implemented through hardware taint tracking

```

1 void vulnerable_code(size_t idx) {
2     if (idx < array_size) { // B1
3         u8 data = array[idx]; // speculative access, execute & taint!
4         u8 X = data + 2; // not a transmitter, execute!
5         if (idx % 2) { // B2
6             u8 junk = glb_array[X*4096]; // transmitter, delay!
7         }
8     }
9 }

```

■ Tainted

Root of taint



When Instruction 3 becomes non-speculative---i.e., B1 resolves---“data” becomes untainted, and Instruction 6 can execute

* Yu et al., “Speculative Taint Tracking (STT): A Comprehensive Protection for Speculatively Accessed Data” (MICRO '20)

Speculative Taint Tracking (STT)

Leakage through implicit channels

Tainted

```
1 void vulnerable_code(size_t idx) {  
2     if (idx < array_size) {  
3         u8 data = array[idx]; // speculative access  
4         if (data & 1) { // B1  
5             div();  
6         } else {  
7             mul();  
8         }  
9     }  
10 }
```

- Speculatively resolve B1 based on the data
⇒ Execution unit usage leaks the data
- Speculatively update BPU entries of B1 based on the data
⇒ Future predictions of B1 leaks the data

Cannot speculatively resolve branches and update BPU tables

Recap: Branch Target Inject (Spectre v2)

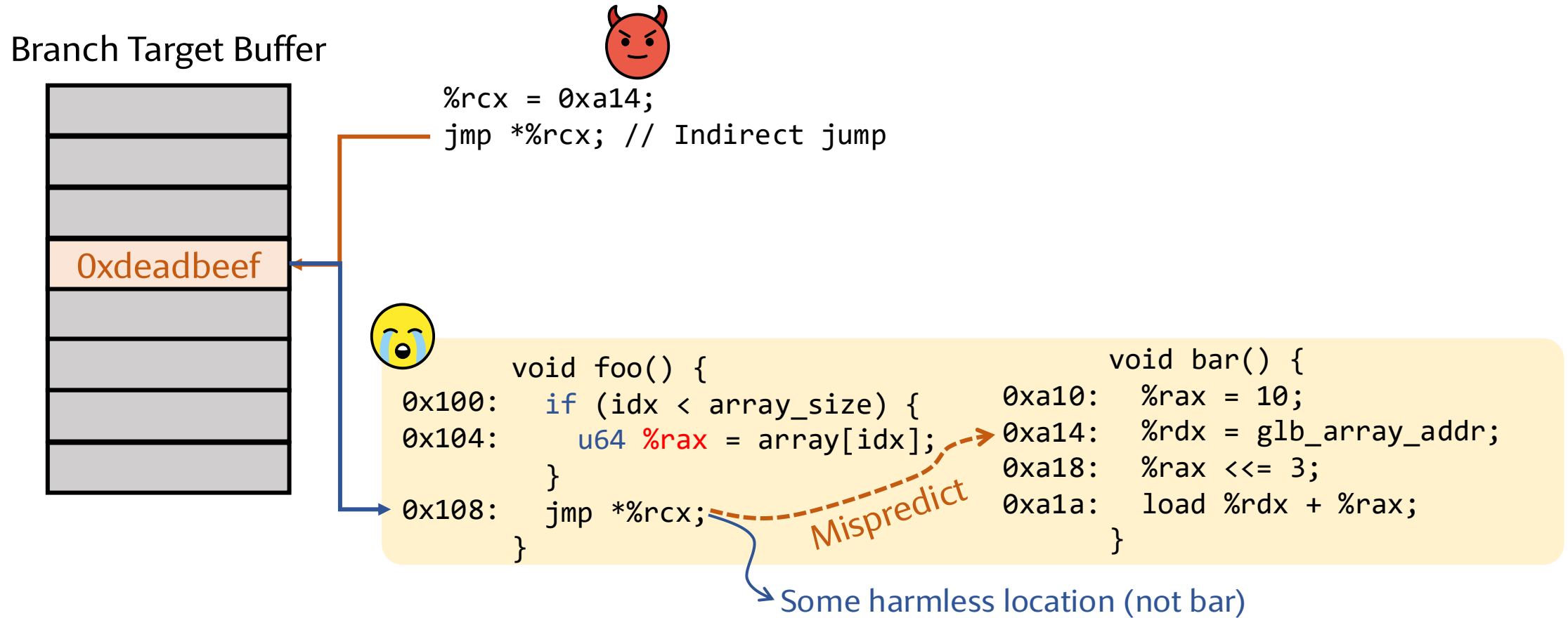
Branch Target Buffer



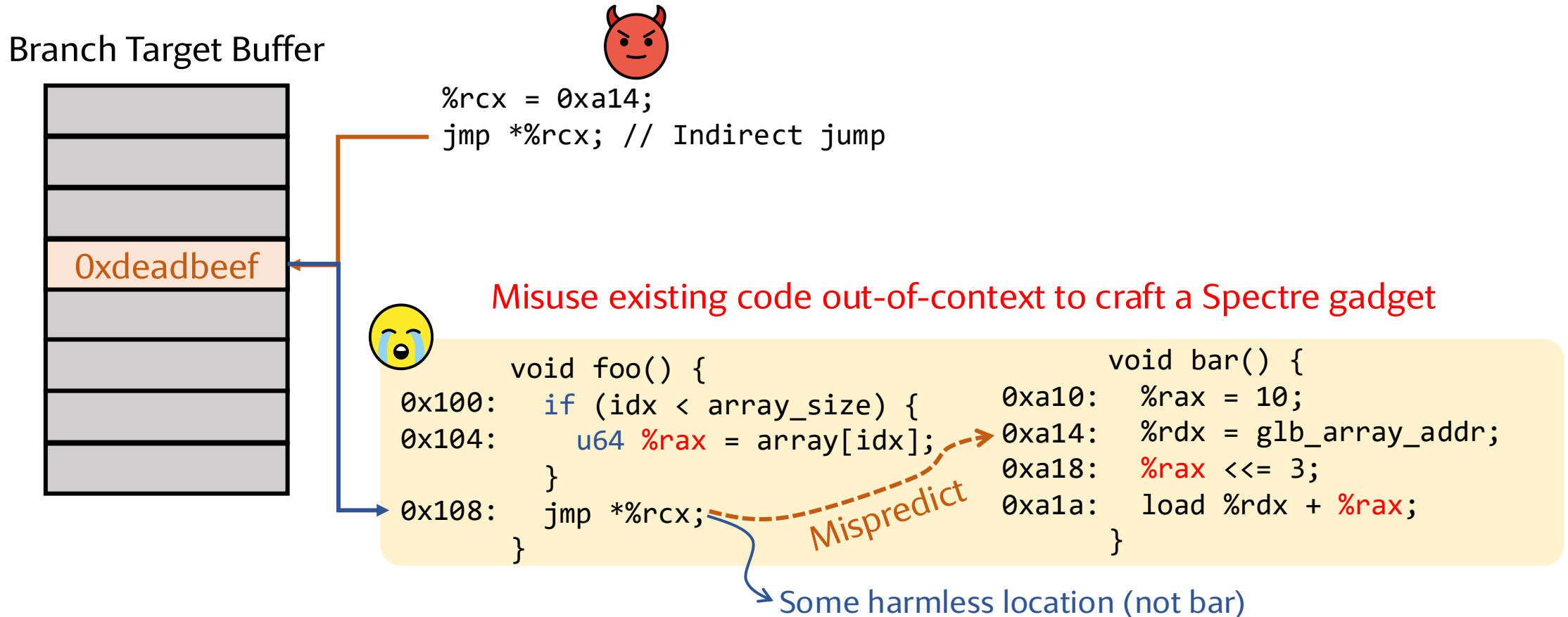
```
%rcx = load addr; // Long latency  
jmp *%rcx; // Indirect jump
```

Jump to "0xdeadbeef"

Recap: Branch Target Inject (Spectre v2)

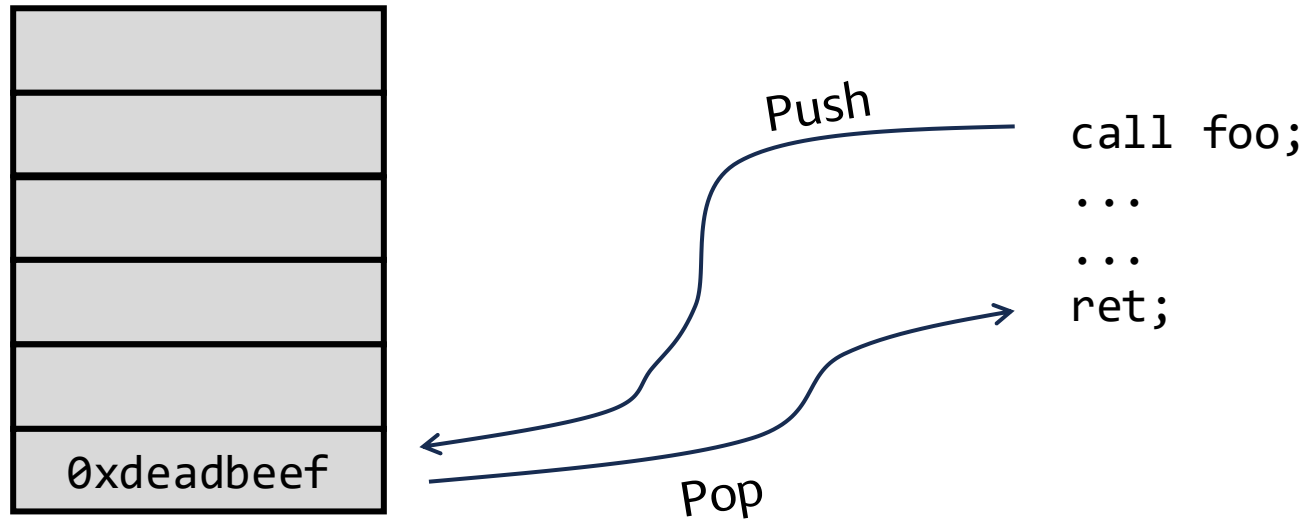


Recap: Branch Target Inject (Spectre v2)



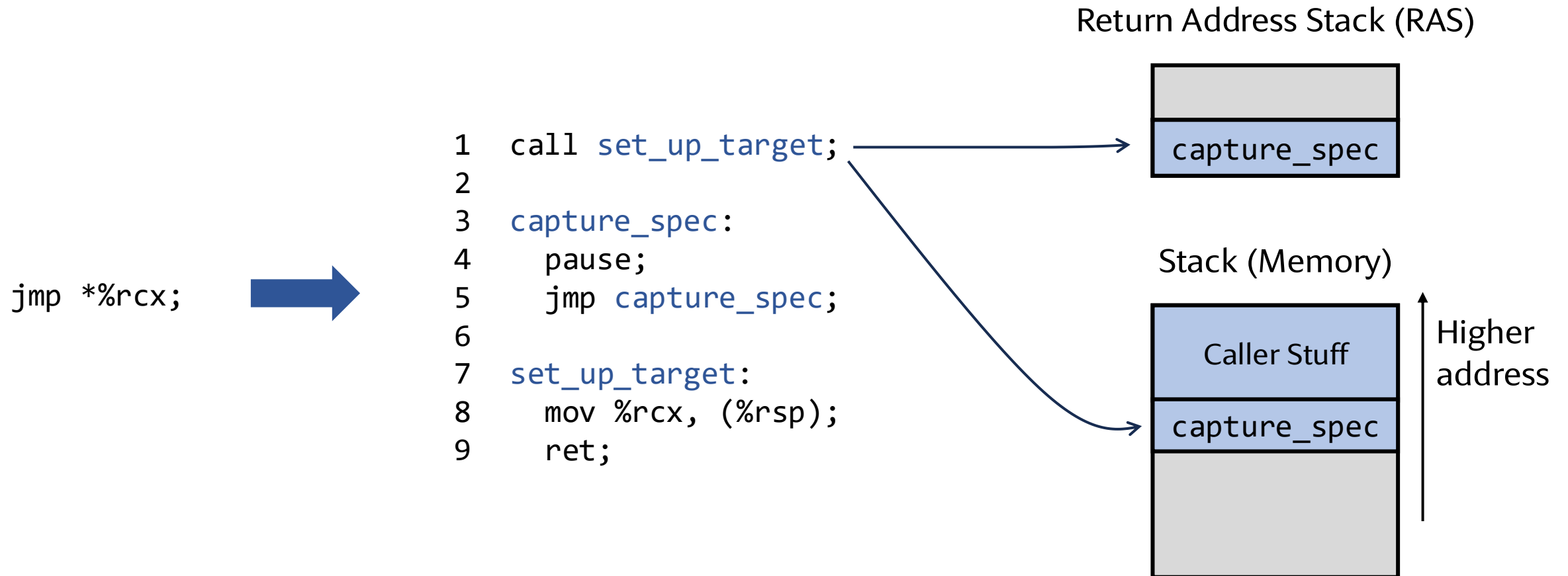
Return Address Stack

Return Address Stack (RAS)

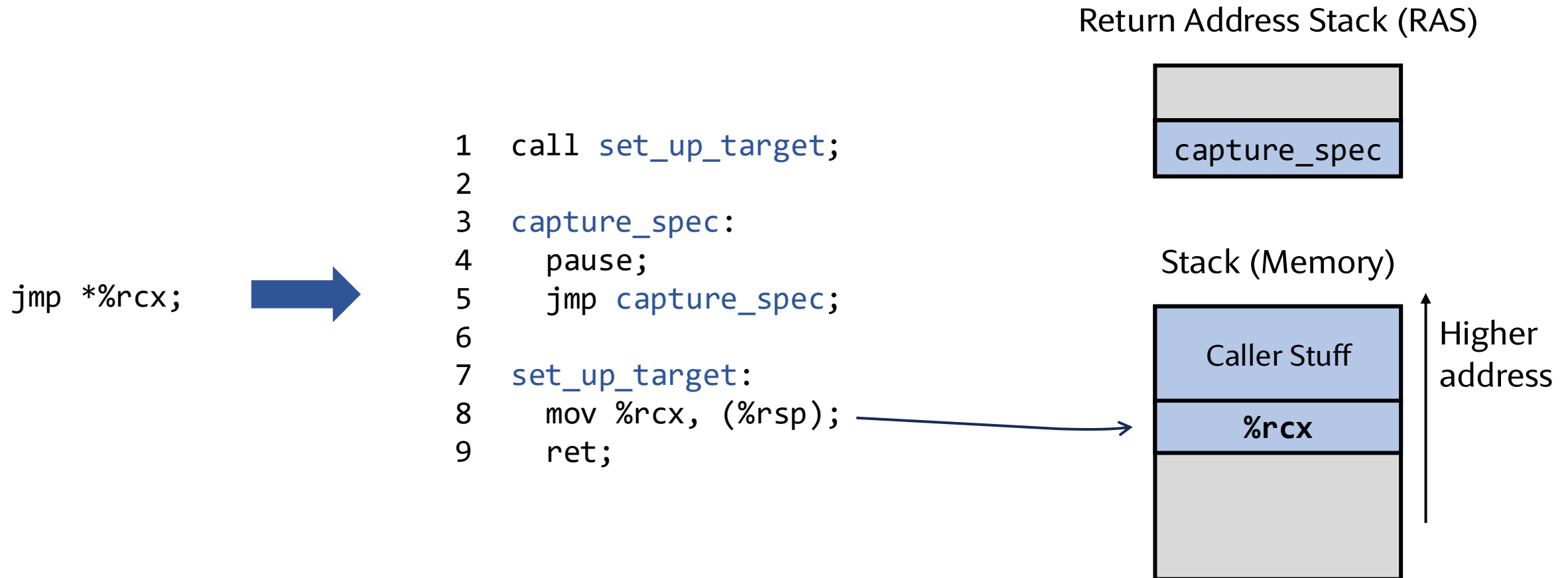


RAS's prediction overwrites BTB's prediction

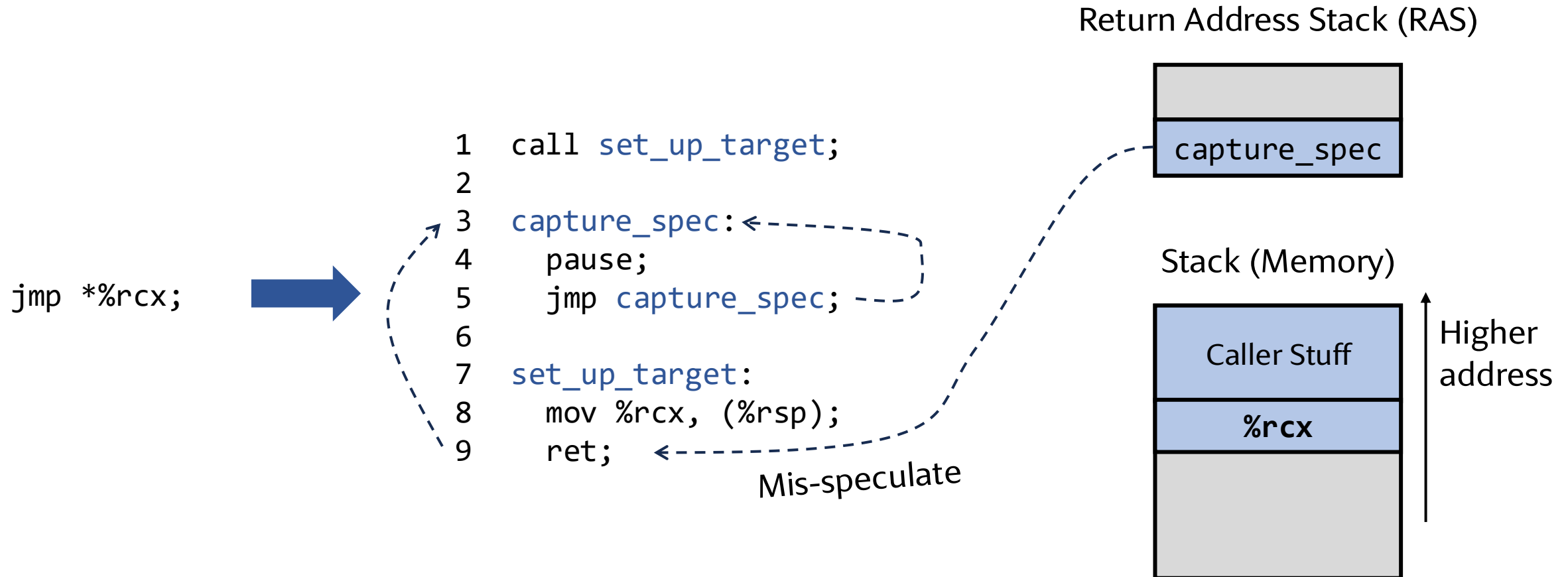
Retpoline: “Poison” Our Own Prediction



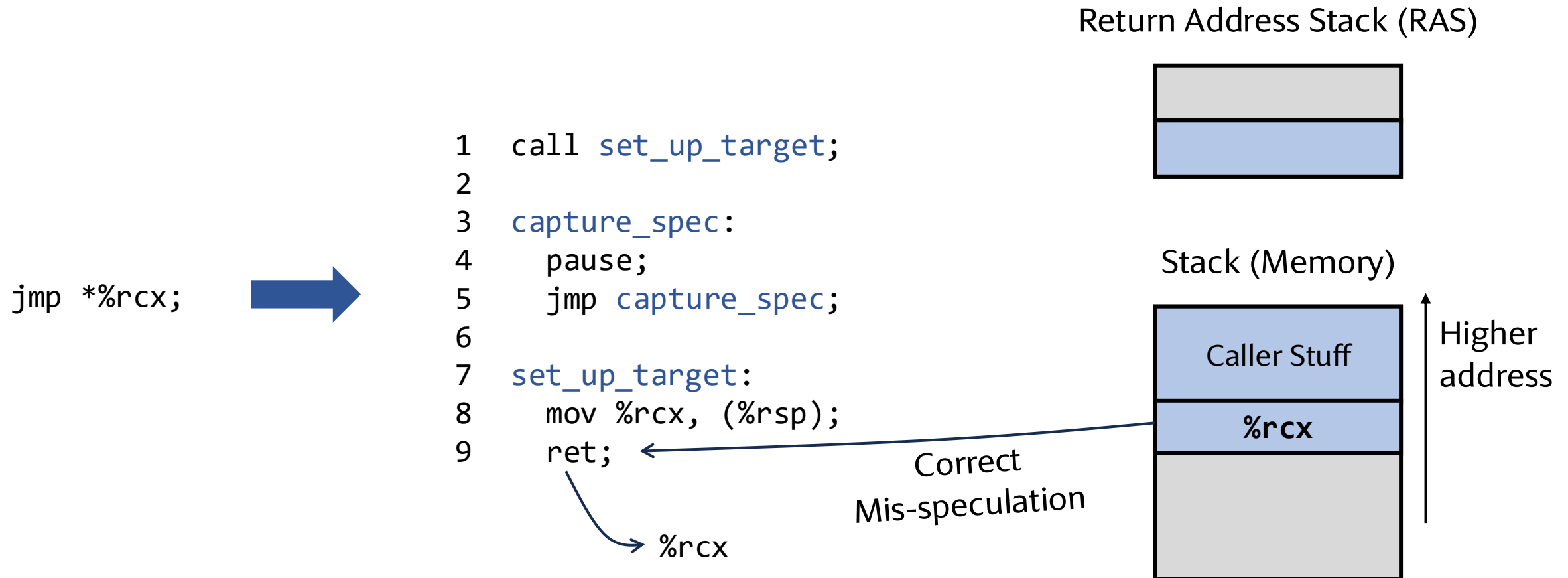
Retpoline: “Poison” Our Own Prediction



Retpoline: "Poison" Our Own Prediction



Retpoline: "Poison" Our Own Prediction



Retpoline: "Poison" Our Own Prediction

